# API ROUTER

## Connect. Manage. Route APIs

Cahier des Charges Technique & Fonctionnel

**KERNEL FORGE**

Version 2.1 • 24 February 2026

| | |
|---|---|
| **Projet** | API ROUTER — Gestionnaire dynamique d'APIs Node.js |
| **Équipe** | KERNEL FORGE |
| **Client** | À définir |
| **Rédacteur** | Équipe Technique — KERNEL FORGE |
| **Version** | 2.1 |
| **Date** | 24/02/2026 |
| **Statut** | En cours de validation |

■ ***Document confidentiel*** *— Destiné exclusivement aux membres de l'équipe KERNEL FORGE et aux parties prenantes autorisées. Reproduction ou diffusion non autorisée interdite.*

# Table des Matières


| | | |
|---|---|---|
| 1. | Contexte et Introduction | 3 |
| 2. | Objectifs du Projet | 3 |
| 3. | Périmètre et Contraintes | 4 |
| 4. | Architecture Technique | 4 |
| 5. | Fonctionnalités Détaillées | 5 |
| 5.1 | Gestion des APIs — Fichier & Dossier | 5 |
| 5.2 | Interface Utilisateur — Frontend ReactJS | 6 |
| 5.3 | Backend — Python Django | 7 |
| 6. | Exigences Techniques Spécifiques | 8 |
| 7. | Sécurité et Performances | 9 |
| 8. | Plan de Tests | 10 |
| 9. | Planning et Livrables | 11 |
| 10. | Glossaire | 11 |

# 1. Contexte et Introduction

Dans un contexte de prolifération des microservices et APIs Node.js, les équipes de développement font face à une complexité croissante : attribution manuelle de ports, absence de point d'entrée unifié, difficulté de supervision. **API ROUTER**, développé par **KERNEL FORGE**, répond à ces défis en offrant une plateforme centralisée de publication, routage dynamique et visualisation d'APIs Node.js, propulsée par un backend Python Django et une interface ReactJS moderne.

Un soin particulier est apporté à la prise en charge des **APIs chargées depuis des dossiers complets** (archive ZIP avec package.json, dépendances, SQLite, .env…), car c'est le cas d'usage le plus courant en développement réel.

# 2. Objectifs du Projet

| Objectif | Description |
|---|---|
| Centralisation | Regrouper et administrer plusieurs APIs Node.js depuis un point de contrôle unique. |
| Déploiement simplifié | Éliminer la gestion manuelle des ports grâce au routage dynamique automatisé. |
| Chargement par dossier | Charger des APIs complètes depuis un dossier projet (package.json, dépendances, assets, SQLite). |
| Accès personnalisable | Définir des préfixes d'URL thématiques (/math, /phy, /auth) par API. |
| Visualisation | Explorer et tester les endpoints via une interface graphique intuitive. |
| Isolation | Garantir l'indépendance totale entre les APIs (ports, fichiers, bases de données, variables d'env). |
| Observabilité | Monitorer l'état de santé, les logs et les métriques de chaque API en temps réel. |

# 3. Périmètre et Contraintes

**Dans le périmètre (In Scope) :**

- Chargement d'APIs depuis un **dossier complet uploadé** (archive ZIP) avec résolution des dépendances npm
- Chargement d'un fichier `server.js` seul (mode simplifié)
- Routage dynamique avec préfixes d'URL personnalisables
- Interface de gestion, visualisation et test des APIs
- Gestion des dépendances locales : SQLite, fichiers statiques, modules relatifs, .env
- Authentification administrateur JWT avec gestion des rôles
- Génération de snippets de code (fetch, axios, curl, Python requests)
- Journalisation temps réel et monitoring de santé des APIs

**Hors périmètre (Out of Scope) :**

- Support d'autres runtimes que Node.js pour les APIs chargées

- Gestion des APIs distantes (uniquement fichiers/dossiers locaux uploadés)
- Fonctionnalités CI/CD intégrées
- Support multi-tenant (prévu en v2)

| Contrainte | Description | Impact |
|---|---|---|
| Sécurité | Isolation stricte des processus Node.js et sandboxing | Critique |
| Performance | Latence proxy ajoutée < 50 ms (p95) | Haut |
| Compatibilité | Node.js >= 16.x LTS requis pour les APIs chargées | Moyen |
| Taille dossier | Archive ZIP max 500 Mo par API (configurable) | Moyen |
| Scalabilité | Maximum 50 APIs Node.js simultanées en v1 | Faible |

# 4. Architecture Technique

API ROUTER adopte une architecture en trois couches séparées. Le Frontend ReactJS communique avec le Backend Django via une API REST sécurisée (JWT). Le Backend agit comme proxy intelligent : il intercepte les requêtes, identifie l'API cible via le préfixe d'URL et transfère la requête vers le processus Node.js isolé correspondant sur son port interne.

| Couche | Technologie | Rôle | Port |
|---|---|---|---|
| Frontend | ReactJS + Vite 5 | UI, dashboards, explorateur d'API | 3000 |
| Backend API | Python Django 4.2 + DRF | Logique métier, routage, proxy, persistance | 8000 |
| WebSocket | Django Channels 4 + Redis | Logs temps réel, notifications | 8001 |
| APIs Node.js | Node.js 18 LTS + Express | APIs chargées dynamiquement | Dyn. |
| Base de données | PostgreSQL 14+ | Persistance interne d'API ROUTER | 5432 |
| Cache | Redis 7+ | Sessions JWT, pub/sub WebSocket | 6379 |
| Isolation APIs | Docker / subprocess | Sandboxing de chaque processus Node.js | — |

**Exemples de flux de requêtes :**

```
GET /math/api/calcul → Django Proxy → Node.js :10042/api/calcul → JSON
POST /user/register → Django Proxy → Node.js :10078/register → JSON
```

# 5. Fonctionnalités Détaillées

## 5.1 Gestion des APIs — Fichier & Dossier

### Mode Principal : Chargement depuis un Dossier Complet

- Upload d'une **archive ZIP** contenant le dossier complet de l'API (drag & drop ou sélection)
- Le dossier doit contenir `server.js` ou le fichier défini dans `package.json > main` à sa racine
- Préservation de l'arborescence complète : `src/`, `routes/`, `middlewares/`, `models/`, `public/`, `uploads/`
- Détection du `package.json` et installation automatique des dépendances npm dans un dossier isolé
- Lecture et injection du fichier `.env` inclus dans le dossier
- Support des modules locaux référencés par chemins relatifs (`require('./utils')`)
- Détection de la version Node.js requise via le champ `engines.node`

### Mode Simplifié : Fichier server.js Unique

- Upload d'un fichier `server.js` seul pour les APIs sans dépendances locales
- Détection automatique des routes Express.js et des middlewares
- Validation syntaxique avant exécution avec rapport d'erreurs clair
- Option d'association ultérieure d'un dossier de ressources

### Gestion des Ressources Locales

- Montage du dossier dans un environnement isolé reproduisant fidèlement l'arborescence d'origine
- Bases de données **SQLite** : chaque API accède à son propre fichier `.db` sans interférence
- Fichiers statiques servis par Express (`express.static`) avec chemins résolus correctement
- Dossier de destination multer mappé dans l'environnement isolé

### Routage Dynamique et Personnalisable

- Définition d'un préfixe d'URL libre par API (ex : /math, /auth, /data)
- Accès final : `http://HOST:PORT/PREFIX/route_originale`
- Modification du préfixe sans redémarrage de l'API
- Détection et blocage des conflits de préfixes avec alerte utilisateur

### Isolation et Conteneurisation

- Chaque API s'exécute dans un **processus enfant isolé** ou un **conteneur Docker léger**
- Attribution dynamique d'un port interne dans une plage configurable (ex : 10000–11000)
- Variables d'environnement distinctes et étanches entre APIs
- Healthcheck périodique et redémarrage automatique en cas de crash

## 5.2 Interface Utilisateur — Frontend ReactJS

### Tableau de Bord Principal

- Vue d'ensemble de toutes les APIs avec statut temps réel (badge : actif / inactif / erreur)
- Indicateurs de performance : temps de réponse moyen, requêtes/min, taux d'erreur
- Filtres par statut, préfixe, date de chargement ; tri par colonne
- Actions rapides : démarrer, arrêter, redémarrer, supprimer une API

### Explorateur d'API (style Swagger UI)

- Liste détaillée des routes disponibles (GET, POST, PUT, DELETE, PATCH)
- Affichage des paramètres : body, query params, path params, headers requis
- Interface de test intégrée : envoi de requêtes depuis le navigateur
- Visualisation des réponses JSON avec coloration syntaxique et formatage

### Gestion des Fichiers et Dossiers

- **Mode dossier :** drag & drop d'une archive ZIP du projet API complet
- **Mode fichier unique :** upload d'un fichier `server.js` standalone
- Barre de progression : upload → extraction → analyse → npm install → démarrage
- Visualisation interactive de l'arborescence avant confirmation du chargement
- Indicateur des dépendances npm détectées et de leur statut d'installation

### Génération de Code d'Appel

- Snippets auto-générés : JavaScript fetch, axios, curl, Python requests
- Copie en un clic dans le presse-papiers
- Pré-remplissage avec les valeurs de test saisies

### Journaux et Monitoring

- Flux de logs temps réel via WebSocket pour chaque API
- Filtrage par niveau (INFO, WARN, ERROR) et par mot-clé
- Historique des requêtes avec détail des paramètres et réponses
- Alertes configurables sur erreurs répétées ou timeouts

## 5.3 Backend — Python Django

### Moteur de Chargement et d'Exécution Node.js

- Module Django dédié à la gestion du cycle de vie des processus Node.js
- Extraction et validation de l'archive ZIP : structure, point d'entrée, package.json
- Installation automatique des dépendances npm (`npm install --prefix /chemin/isolé`)
- Lancement via subprocess avec capture des flux stdout/stderr
- Healthcheck périodique (HTTP ping sur le port interne) et redémarrage automatique

### Parseur de Routes Express.js (Analyse AST)

- Analyse AST du code JavaScript via **esprima** ou **acorn**
- Extraction des routes, méthodes HTTP, chemins, middlewares (multer, cors, auth)
- Identification des dépendances npm et de la version Node.js requise
- Génération automatique du schéma **OpenAPI 3.0**
- Détection des modules locaux (`require('./...')`) pour cartographier les dépendances internes

### Système de Proxy HTTP Asynchrone

- Proxy transparent basé sur **httpx** (client async Python)
- Gestion du timeout, des retries (3 tentatives) et des circuit breakers
- Réécriture des URLs : suppression du préfixe avant transmission
- Transmission des headers d'origine + ajout de X-Forwarded-For

### Authentification et Autorisation

- Authentification JWT pour la communication frontend ↔ backend
- Rôles : **Admin** (accès complet) et **Viewer** (lecture seule)
- Tokens de rafraîchissement avec expiration configurable
- Audit log : traçabilité complète des actions administrateur

## 6. Exigences Techniques Spécifiques

| Composant | Technologie | Version min. | Justification |
|---|---|---|---|
| Runtime Backend | Python | 3.11+ | Performances async, type hints |
| Framework Backend | Django + DRF | 4.2 / 3.14+ | LTS, API REST robuste |
| Runtime APIs | Node.js LTS | 18.x | Stabilité et compatibilité Express |
| Framework Frontend | ReactJS + Vite | 18+ / 5+ | Performance, HMR rapide |
| Base de données | PostgreSQL | 14+ | Fiabilité, support JSON natif |
| Cache / WebSocket | Redis | 7+ | Sessions JWT, pub/sub temps réel |
| Isolation APIs | Docker / subprocess | 24+ / stdlib | Sandboxing des processus Node.js |
| Proxy HTTP async | httpx | 0.25+ | Client async performant |
| Analyse AST JS | esprima / acorn | latest | Parsing JavaScript côté Python |
| WebSocket backend | Django Channels | 4+ | Logs en temps réel |
| Gestion archives | Python zipfile | stdlib | Extraction des dossiers ZIP |
| Uploads dans APIs | multer | latest | Gestion des fichiers dans les APIs |

### Variables d'environnement requises (.env backend) :

```
SECRET_KEY=
DEBUG=False
DATABASE_URL=postgresql://user:password@localhost:5432/apirouter
REDIS_URL=redis://localhost:6379/0
NODE_BINARY=/usr/bin/node
NPM_BINARY=/usr/bin/npm
API_PORT_RANGE_START=10000
API_PORT_RANGE_END=11000
API_STORAGE_PATH=/var/apirouter/apis
UPLOAD_MAX_SIZE_MB=500
NPM_INSTALL_TIMEOUT=120
JWT_ACCESS_TOKEN_LIFETIME=60
MAX_CONCURRENT_APIS=50
```

# 7. Sécurité et Performances

## 7.1 Exigences de Sécurité

- **Isolation des processus :** chaque API Node.js s'exécute sous un utilisateur système dédié avec droits réduits (least privilege).
- **Validation des archives :** vérification du contenu ZIP avant extraction (path traversal, taille, extensions autorisées).
- **Liste blanche npm :** validation optionnelle des dépendances déclarées dans package.json avant installation.
- **Sandbox réseau :** les processus Node.js n'accèdent qu'à leur port interne alloué.
- **CORS :** configuration stricte des origines autorisées, configurable par API.
- **HTTPS :** support TLS natif avec génération de certificats auto-signés pour les environnements de développement.
- **Rate Limiting :** limitation du nombre de requêtes par IP et par API.
- **Audit log :** traçabilité complète des actions administrateur (chargement, arrêt, suppression).

## 7.2 Objectifs de Performance (SLOs)

| Métrique | Objectif (cible) | Seuil critique |
|---|---|---|
| Latence proxy ajoutée | < 50 ms (p95) | < 100 ms |
| Temps de démarrage d'une API | < 5 secondes | < 10 secondes |
| Extraction ZIP + npm install | < 60 secondes | < 120 secondes |
| APIs Node.js simultanées | 50 (v1) | 100 (v2) |
| Disponibilité du service | 99,5 % | 99 % |
| Temps de réponse UI | < 200 ms (p95) | < 500 ms |

## 8. Plan de Tests

| ID | Catégorie | Description | Critère de succès | Priorité |
|---|---|---|---|---|
| T-01 | Dossiers | Upload ZIP complet avec package.json et dépendances | npm install OK, API démarrée | Critique |
| T-02 | Dossiers | API avec modules locaux (require relatif) | Résolution correcte des chemins | Critique |
| T-03 | Dossiers | ZIP sans server.js ni main dans package.json | Message d'erreur explicite | Critique |
| T-04 | Dossiers | Archive ZIP malformée ou corrompue | Rejet propre avec alerte | Haut |
| T-05 | Dossiers | API utilisant un fichier SQLite local | Accès aux données correct | Critique |
| T-06 | Dossiers | API utilisant un fichier .env inclus | Variables d'env correctement chargées | Haut |
| T-07 | Fichier | Chargement d'un server.js valide standalone | API démarrée et accessible | Critique |
| T-08 | Fichier | Chargement d'un server.js invalide (syntaxe) | Message d'erreur clair | Critique |
| T-09 | Routage | Accès via préfixe personnalisé | Réponse correcte reçue | Critique |
| T-10 | Routage | Conflit de préfixes entre deux APIs | Alerte et refus de chargement | Haut |
| T-11 | API | Requêtes GET, POST, PUT, DELETE vers API chargée | Réponses conformes aux specs | Critique |
| T-12 | Isolation | Deux APIs avec même nom de route | Aucune interférence entre APIs | Critique |
| T-13 | UI | Visualisation des routes et paramètres | Affichage complet et exact | Moyen |
| T-14 | UI | Génération de snippets de code | Code exécutable et fonctionnel | Moyen |
| T-15 | Perf. | Latence proxy sous charge (100 req/s) | P95 < 50 ms | Haut |
| T-16 | Sécurité | Upload d'un ZIP avec path traversal | Extraction bloquée | Critique |
| T-17 | Robustesse | Crash d'une API Node.js en cours d'exécution | Redémarrage auto ou alerte claire | Haut |

## 9. Planning et Livrables

| Phase | Durée est. | Livrables principaux |
|---|---|---|
| Phase 0 — Cadrage & Setup | 1 semaine | CDC validé, choix technologiques, dépôt Git initialisé |
| Phase 1 — Backend Core | 3 semaines | Modèles Django, API REST, moteur d'exécution Node.js, proxy async |
| Phase 2 — Dossiers & AST | 2 semaines | Upload ZIP, extraction, npm install isolé, parseur AST, OpenAPI |
| Phase 3 — Frontend ReactJS | 3 semaines | Dashboard, explorateur d'API, gestionnaire de dossiers, snippets |
| Phase 4 — Intégration | 1 semaine | Tests d'intégration, hardening sécurité, corrections |
| Phase 5 — Tests & Livraison | 1 semaine | Plan de tests complet, documentation, déploiement, formation |

*Durée totale estimée : **11 semaines**. Ce planning sera affiné lors du cadrage avec l'équipe KERNEL FORGE.*

## 10. Glossaire

| Terme | Définition |
|---|---|
| `server.js` | Fichier Node.js définissant un serveur Express.js avec ses routes et middlewares. |
| `Express.js` | Framework web minimaliste pour Node.js, utilisé pour définir les routes des APIs. |
| `Archive ZIP` | Format de compression utilisé pour uploader un dossier complet d'API. |
| `package.json` | Fichier manifeste Node.js décrivant les métadonnées du projet et ses dépendances npm. |
| `node_modules` | Dossier des dépendances npm installées, généré lors du npm install. |
| `Proxy` | Intermédiaire qui redirige les requêtes vers la destination appropriée. |
| `AST` | Abstract Syntax Tree : représentation arborescente du code source pour l'analyse statique. |
| `DRF` | Django REST Framework : extension Django pour la création d'APIs REST. |
| `JWT` | JSON Web Token : standard d'authentification basé sur des tokens signés cryptographiquement. |
| `SQLite` | Base de données relationnelle légère stockée dans un fichier unique. |
| `CORS` | Cross-Origin Resource Sharing : mécanisme HTTP contrôlant les accès cross-origine. |
| `SLO` | Service Level Objective : objectif de niveau de service mesurable (disponibilité, latence…). |
| `multer` | Middleware Express.js pour la gestion des uploads de fichiers multipart/form-data. |
| `KERNEL FORGE` | Équipe de développement en charge du projet API ROUTER. |

*Cahier des Charges — API ROUTER v2.1 | KERNEL FORGE | 24/02/2026 | Toute modification doit faire l'objet d'un avenant validé par les deux parties.*